

Package ‘twostageGWASsurvival’

July 3, 2024

Title Implement a Two Stage Method for Detecting Epistasis in GWAS for Survival Data

Version 0.5.1

Encoding UTF-8

Author Luc van Schijndel [aut, cre],
Marianne Jonker [aut]

Maintainer Luc van Schijndel, <lcvsch@hotmail.com>

Description Using a two stage method, it is possible to reduce the computational complexity of performing a GWAS which aims to detect epistasis between two SNPs. The total computation time is on the scale of only days instead of on the scale of centuries, as is the case for a naive approach. This package implements a function for applying this method to data obtained for example in a GWAS. It is recommended to perform this method on a machine capable of large parallel computations.

License GPL (>= 3)

RoxygenNote 7.3.1

Suggests devtools,
doParallel,
knitr,
LDheatmap,
rmarkdown,
testthat (>= 3.0)

Imports stats, utils, Matrix, survival, foreach

Depends R (>= 3.5)

LazyData true

VignetteBuilder knitr

URL <https://github.com/luc-van-schijndel/twostageGWASsurvival>

BugReports <https://github.com/luc-van-schijndel/twostageGWASsurvival/issues>

Contents

batched.twostagecoxph	2
example_snp_data	5
example_survival_data	5
print.twostageGWAS	6
twostagecoxph	7
twostagecoxph.control	10

Index**11**

batched.twostagecoxph *Performs a large scale two stage analysis on a survival dataset to detect epistasis using batches*

Description

An adaptation of [twostagecoxph](#) where the covariates will be read from multiple files. This is to alleviate possible memory constraints. The main functionality assumes a parallel back-end is registered, e.g. using `doParallel::registerDoParallel(2)`.

The theory supporting the statistical correctness of this function is described in [this](#) paper by Jonker et al.

Usage

```
batched.twostagecoxph(
  survival.dataset,
  covariate.filepaths,
  first.stage.threshold = 0.05,
  multiple.hypotheses.correction = "bonferroni",
  updatefile = "",
  control = twostagecoxph.control(),
  number.of.covariates = 0,
  read.function = function(x) as.matrix(read.table(x))
)
```

Arguments

survival.dataset	The survival dataset describing the outcome.
covariate.filepaths	The vector of paths of the files containing the covariates. See Details.
first.stage.threshold	Numeric scalar denoting the threshold for the first stage. If a covariate has a p-value lower than this threshold, it will be passed on to the second stage.
multiple.hypotheses.correction	Correction method, a character string. Passed to p.adjust . Possible methods are given by p.adjust.methods .
updatefile	Path to a text file where updates may be written. Necessary for parallel computations, since the connection to the terminal will be lost. This file will in that case serve as a stand-in for the terminal. The default equals a connection to the terminal which is lost when a parallel back-end is registered.
control	Object of class twostagecoxph.control specifying various options for performance of the two stage method.
number.of.covariates	The combined total of covariates found in the files specified. If left unspecified, the function will automatically determine this value by reading through all files before starting the analysis, thereby increasing runtimes.

`read.function` Function used to read batches of covariates from the files specified. By default, this is `function(x) as.matrix(utils::read.table(x))`. Note that the first argument of the function will be used to pass the filepaths. See Details.

Details

At all times on all registered cores, the contents of at most 2 files specified by `covariate.filepaths` will be in memory. Take note however, that the results will be written to a matrix of substantial size (`covariates_per_file` x `total_number_of_covariates`) so it is best if enough memory is still available after reading the files.

The function assumes that the object returned by `read.function` is a matrix containing the covariates with one column per covariate and one row per subject. This assumption will only be checked once at function start. If the files are not structured the same, the results may be unreliable or an error may be thrown. If the `read.function` assigns names to the columns via the `dimnames` attribute, these names will be kept and passed on to the resulting object. If not, the indices of the covariates (counting across all files) will be used as the names.

The function assumes that the files have the same number of covariates, except for the last one, which may have less. `number.of.covariates` will be divided by the amount of covariates per file, and the remainder of that division *must* be the number of covariates in the last file.

Value

A `twostageGWAS` object, which is a list of 7 entries:

`result.list` A list containing the results and where to find them in either 5 or 7 entries, depending on whether or not the covariates are named in the files:

- `p.values` the p-values of the interactions not equal to 1. The list is sorted in ascending order by this value. Any p-values that are either NA, 0, or 1 after possibly applying the multiple hypotheses correction will not be present in this vector.
- `batch.one, index.one, batch.two, index.two` the indices describing where to find the two covariates that describe the interaction corresponding to the p-values in the previous entry. `batch.one` and `batch.two` give the indices of the files as described in `covariate.filepaths` containing the two covariates. `index.one` and `index.two` give the indices of the covariates *within* those files, i.e. the local indices.
- `names.one, names.two` if the covariates are named, these are the names of the covariates found on the aforementioned indices.

`most.significant.results` A list describing the most significant results found used primarily by the print method. The number of results reported is specified by the control parameter, default 5. The list contains 3 items:

- `interacting.snps` the names of the interaction, in `name_snp_1` x `name_snp_2` format. In case the covariates in `covariate.matrix` have a `names` attribute, these names are used. Otherwise the index of the covariates within the matrix is used.
- `p.value.epistasis` the corresponding p-values of the interactions. Unless `return.raw = TRUE` is specified in the control parameter, these p-values will be corrected for the multiple hypotheses tested with the method specified by the `multiple.hypotheses.correction` parameter.

<code>duplicate.interactions</code>	the list of duplicate interactions found, corresponding to the interactions specified in this list. An interaction is said to be duplicate if the corresponding p-value is the same up until 7 significant figures.
<code>p.value.matrix</code>	A <code>sparseMatrix</code> from the package <code>Matrix</code> , specifying the resulting upper triangular p-value matrix obtained from the second stage. Unless <code>return.raw = TRUE</code> is specified in the control parameter, these p-values will be corrected for the multiple hypotheses tested with the method specified by the <code>multiple.hypotheses.correction</code> parameter. The names of the dimensions of the matrix match the ones specified in the files, if the <code>read.function</code> assigns <code>dimnames</code> to the matrix. The row and columns corresponding to covariates that were not passed to the second stage, have the string <code>as.character(NA)</code> as names (Note: not <code>NA</code> itself). The entries in these rows and columns are also all absent.
<code>marginal.significant</code>	A vector of named integers, specifying the indices of covariates which were found to be marginally significant in the first stage.
<code>first.stage</code>	A vector specifying the p-values found in the first stage. These p-values are <i>not</i> corrected for the multiple tested hypotheses.
<code>fst</code>	The threshold for significance used in the first stage as specified in <code>first.stage.threshold</code>
<code>runtime</code>	The total runtime of the function in seconds.
<code>call</code>	The matched call.

Note

Parallel processing requires a properly registered parallel back-end, such as one obtained from `doParallel::registerDoParallel(2)` to be used by `foreach` and the `%dopar%` binary operator.

Be aware that any progress updates are only rough estimates of the current progress and remaining runtime. Since the parallel processes are not inter-connected, the estimates are based on the progress itself and therefore highly unreliable if the fraction of processes to workers is relatively low, especially on non-Unix like operating systems.

See Also

[Paper by Jonker et al.](#), [foreach](#), [print.twostageGWAS](#), [twostagecoxph.control](#)

Examples

```
str(example_survival_data)
str(example_snp_data)

#We shrink the example to run faster, try altering this! :)
example_snp_data <- example_snp_data[,1:100]

#Split the covariate matrix into various files.
number.of.covs <- dim(example_snp_data)[2]
number.of.files <- 6
# 500 is not a multiple of 6, so the last file has less covariates than the other ones
temp.snpfile.paths <- tempfile(rep("snpfile", number.of.files),
                                tmpdir = tmpdir(check = TRUE),
                                fileext = ".txt")

indices.matrix <-
  matrix(c(seq_len(number.of.covs),
```

```

      rep(NA, ceiling(number.of.covs/number.of.files)*number.of.files - number.of.covs)),
      ncol = number.of.files)
for(file.num in 1:number.of.files){
  indices <- indices.matrix[,file.num]
  write.table(example_snp_data[,indices[!is.na(indices)]], file = temp.snpfile.paths[file.num])
}

#make a file to get updates on the progress of the function
updatefile <- tempfile("updatefile", fileext = ".txt")
file(updatefile)

foo <- batched.twostagecoxph(example_survival_data, temp.snpfile.paths,
                             number.of.covariates = number.of.covs,
                             first.stage.threshold = 1e-2,
                             updatefile = updatefile)

readLines(updatefile)
print(foo)

#Clean up files from this example
unlink(temp.snpfile.paths)
unlink(updatefile)

```

example_snp_data

Simulated example dataset of SNPs obtained during a GWAS

Description

A dataset containing the numeric interpretation of 500 SNPs from 200 simulated humans. Both dimensions have a number-based names attribute.

Usage

```
example_snp_data
```

Format

A matrix of 500 rows and 200 columns with integer values being either 0, 1 or 2.

Examples

```

if(requireNamespace("LDheatmap", quietly = TRUE)){
  LDheatmap::LDheatmap(cor(example_snp_data), add.map = FALSE, color = heat.colors(20),
                        title = "Correlation structure example_snp_data")
}

```

example_survival_data *Simulated example dataset for a survival GWAS*

Description

A dataset containing time-to-event outcomes for 200 subjects, simulated from a proportional hazards model with one interaction increasing the hazard. The censoring times have a uniform distribution.

Usage

```
example_survival_data
```

Format

A Surv-class object with 200 rows and 2 variables:

time the time until the event took place

status indicator whether or not the subject was censored. 1 if the event was observed.

print.twostageGWAS *Print a twostageGWAS object*

Description

Print a twostageGWAS object

Usage

```
## S3 method for class 'twostageGWAS'
print(x, ...)
```

Arguments

x object of class twostageGWAS
 ... optional arguments passed on to cat and print.default functions.

Details

Prints a brief overview of the most relevant results (max 5) concluded from the two stage analysis. Includes the original call, the number of covariates deemed statistically significant in the first stage and how many tests will be performed in the second stage. The last table shows the most significant results, ordered from lowest p-value to highest. The name of the interaction is given in the form name_cov_1 x name_cov_2 along with the corresponding p-value and possible duplicates. The duplicates are interactions with the same resulting p-value up to 7 significant digits.

Value

The twostageGWAS object invisibly

Examples

```

survival.dataset <- survival::Surv(c(5,5,3,3,2,2,2,1,1,1),
                                   c(0,0,1,1,1,1,1,1,1,1))
covariate.matrix <- matrix(c(2,2,1,
                             2,2,1,
                             1,2,1,
                             2,1,1,
                             1,1,1,
                             1,1,1,
                             1,0,0,
                             0,1,0,
                             1,0,0,
                             0,0,0),
                           nrow = 10, ncol = 3, byrow = TRUE)
print(twostagecoxph(survival.dataset, covariate.matrix,
                    control = twostagecoxph.control(progress = 0)))

```

twostagecoxph	<i>Perform a two stage analysis on a survival dataset to detect interactions assuming proportional hazards</i>
---------------	--

Description

Performs a two stage analysis to find possible interactions influencing the time to event assuming a Cox proportional hazards model. Most useful in cases where the covariates far outnumber the subjects, e.g. in a Genome Wide Association Study. The function is based on two stages, wherein the first one, all covariates are screened marginally for possible effects. Marginally significant covariates are passed on to the second stage, where they are tested in pairs for an interaction effect.

The theory supporting the statistical correctness of this function is described in [this](#) paper by Jonker et al.

Usage

```

twostagecoxph(
  survival.dataset,
  covariate.matrix,
  first.stage.threshold = 0.05,
  multiple.hypotheses.correction = "bonferroni",
  multicore = FALSE,
  updatefile = "",
  control = twostagecoxph.control(),
  ...
)

```

Arguments

survival.dataset
The survival dataset describing the outcome.

covariate.matrix
The nxp-matrix of covariates of the p covariates of the n patients. Named dimensions are supported.

<code>first.stage.threshold</code>	numeric scalar denoting the threshold p-value for the first stage. If a covariate is marginally more significant than this threshold, it will be passed on to the second stage.
<code>multiple.hypotheses.correction</code>	Correction method, a character string. Passed to <code>p.adjust</code> .
<code>multicore</code>	logical, default FALSE; whether or not the function should use multiple cores in its calculations. See Details.
<code>updatefile</code>	A connection to a text file where updates concerning the execution of the function may be written. Necessary for parallel computations, since the connection to the terminal will be lost. This file will in that case serve as a stand-in for the terminal. Default equals a connection to the terminal which will be lost when a parallel back-end is registered.
<code>control</code>	object of class <code>twostagecoxph.control</code> specifying various options for performance of the two stage method.
<code>...</code>	other arguments to be passed to all calls to <code>coxph()</code> in this function.

Details

It is shown by [Jonker et al.](#) that the two stages are independent of each other. This results in proper control of rate of type I errors by the multiple hypotheses correction method. The power is also increased compared to a naive method, due to the fact that less hypotheses are tested in the second stage resulting in a less strict correction. The main advantage is that only a fraction of the possible interactions is tested, resulting in an enormous decrease in computation times.

If `multicore` is TRUE, the function assumes a proper parallel back-end is registered, e.g. one obtained from `doParallel::registerDoParallel(2)`, to be used by the `foreach` package & functions and `%dopar%`.

If memory constraints become an issue, `batched.twostagecoxph` is available. This function gives the user control in which parts of the set of covariates will be in active memory, allowing for better memory management. This does require the user to make its own partition of the covariates into separate files.

Value

An object of class `twostageGWAS`, which is a list of 7 entries:

<code>result.list</code>	A list containing the results and where to find them in either 3 or 5 entries, depending on whether or not the covariates are named in the files: <code>p.values</code> The non-trivial p-values of the interactions. The list is sorted in ascending order by this value. Any p-values that are either NA, 0, or 1 after possibly applying the multiple hypotheses correction will not be present in this vector. <code>index.one</code>, <code>index.two</code> the indices describing the two covariates that describe the interaction corresponding to the p-values in the previous entry. <code>names.one</code>, <code>names.two</code> if the covariates are named, these are the names of the covariates found on the aforementioned indices.
<code>most.significant.results</code>	A list describing the most significant results found. The number of results reported is specified by the control parameter, default 5. The list contains 3 items:

<code>interacting.snps</code>	the names of the interaction, in <code>name_snp_1</code> x <code>name_snp_2</code> format. In case the covariates in <code>covariate.matrix</code> have a <code>names</code> attribute, these names are used. Otherwise the index of the covariates within the matrix is used.
<code>p.value.epistasis</code>	the corresponding p-values of the interactions. Unless <code>return.raw = TRUE</code> is specified in the control parameter, these p-values will be corrected for the multiple hypotheses tested with the method specified by the <code>multiple.hypotheses.correction</code> parameter.
<code>duplicate.interactions</code>	the list of duplicate interactions found, corresponding to the interactions specified in this list. An interaction is said to be duplicate if the corresponding p-value is the same up until 7 significant figures.
<code>p.value.matrix</code>	A <code>sparseMatrix</code> object from the package <code>Matrix</code> , specifying the resulting upper triangular p-value matrix obtained from the second stage. Unless <code>return.raw = TRUE</code> is specified in the control parameter, these p-values will be corrected for the multiple hypotheses tested with the method specified by the <code>multiple.hypotheses.correction</code> parameter. The names of the dimensions of the matrix match the ones specified in the files, if the <code>read.function</code> applied to those files assigns <code>dimnames</code> to the matrix. The row and columns corresponding to covariates that were not named in the input matrix, have the string <code>as.character(NA)</code> for names (Note: not <code>NA</code> itself).
<code>marginal.significant</code>	A vector of named integers, specifying the indices of covariates which were found to be marginally significant in the first stage.
<code>first.stage</code>	A vector specifying the p-values found in the first stage. These p-values are not corrected for the multiple tested hypotheses.
<code>fst</code>	The threshold for significance used in the first stage as specified in <code>first.stage.threshold</code>
<code>runtime</code>	The total runtime of the function in seconds.
<code>call</code>	The matched call.

Note

Parallel processing requires a properly registered parallel back-end, such as one obtained from `doParallel::registerDoParallel(2)` to be used by `foreach` and the `%dopar%` binary operator.

Be aware that in the case of parallel computations, any progress updates are only rough estimates of the current progress and remaining runtime. Since the parallel processes are not inter-connected, the estimates are based on the progress itself and therefore highly unreliable if the fraction of processes to workers is relatively low, especially on non-Unix like operating systems.

See Also

[Paper by Jonker et al.](#), [foreach](#), [print.twostageGWAS](#), [twostagecoxph.control](#)

Examples

```
survival.dataset <- survival::Surv(c(5,5,3,3,2,2,2,1,1,1),
                                   c(0,0,1,1,1,1,1,1,1,1))
covariate.matrix <- matrix(c(2,2,1,
                             2,2,1,
                             1,2,1,
                             2,1,1,
```

```

      1,1,1,
      1,1,1,
      1,0,0,
      0,1,0,
      1,0,0,
      0,0,0),
      nrow = 10, ncol = 3, byrow = TRUE)
twostagecoxph(survival.dataset, covariate.matrix,
              control = twostagecoxph.control(progress = 0))

str(example_survival_data)
str(example_snp_data)

print(foo <- twostagecoxph(example_survival_data, example_snp_data[,1:200],
                          first.stage.threshold = 5e-5))
print(bar <- twostagecoxph(example_survival_data, example_snp_data[,1:200],
                          first.stage.threshold = 5e-4))
# The [,1:200] subsetting is added to speed up the example. Try removing it! :)

# As we can see, foo and bar have different results. A lower FST generally gives more power, but it
# it risks the possibility to be too strict and consequently *decreasing* power.

```

twostagecoxph.control *Ancillary arguments for controlling the operation of the two stage method using twostagecoxph*

Description

This is used to set various numeric parameters controlling the operation of the two stage method performed by twostagecoxph. Typically it would only be used in a call to twostagecoxph.

Usage

```

twostagecoxph.control(
  report.lowest.amount = 5,
  return.raw = FALSE,
  progress = 1000,
  max.coef = 5,
  max.batchsize = 1000,
  lower.bound.variance = 0.1,
  upper.bound.correlation = 0.9
)

```

Arguments

report.lowest.amount	integer, default 5; denotes how many of the most significant interactions the function should report. Altering this value does not affect the maximum amount reported by the print.twostageGWAS function.
return.raw	logical, default FALSE; whether or not the output should contain the raw p-values or the multiple hypotheses corrected p-values.

progress	numeric, default 1000; how many iterations should pass silently until an update is given about runtime and progress until completion of stages. Set to 0 for no output.
max.coef	numeric, default 5; maximum value for all coefficients in the fitted models. If any are larger than this (in absolute value), then the model rejects them and ignored in further analysis. Can be used to exclude unrealistically large values.
max.batchsize	maximum size of one batch, default 1000; for parallel computing, should be lowered if issues arise concerning memory. Can be raised to slightly increase performance
lower.bound.variance	numeric scalar in the interval $[0, 1)$, default 0.1; the lower bound on the variance of a covariate in the first stage. If the variance is lower, the model using this covariate is not fitted and the covariate is rejected by default.
upper.bound.correlation	numeric scalar in the interval $(0, 1]$, default 0.9; the upper bound on the correlation between two covariates and their interaction. If exceeded (in absolute value), the model using these two correlated variables is not fitted and the interaction between these is rejected by default.

Value

a list containing the values of each of the above constants.

Index

* datasets

example_snp_data, [5](#)

example_survival_data, [5](#)

batched.twostagecoxph, [2](#), [8](#)

connection, [7](#)

example_snp_data, [5](#)

example_survival_data, [5](#)

foreach, [4](#), [9](#)

p.adjust, [2](#), [7](#)

p.adjust.methods, [2](#)

print.twostageGWAS, [4](#), [6](#), [9](#)

twostagecoxph, [2](#), [7](#)

twostagecoxph.control, [2](#), [4](#), [7](#), [9](#), [10](#)